



**SAKARYA
UYGULAMALI BİLİMLER
ÜNİVERSİTESİ**

2025 - 2026 | BAHAR

PROJE-2

Öğrencilerin

Adı Soyadı : Umutcan Yıldız, Enes Doğru, Nihat Can Akkaya, Süreyya Sönmez

Okul No : 23010903072, 23010903048, 22010903092, 23010903078

Proje Başlığı : PicoRV İşlemci Alt Kümesi(RV32I) için Linker Tasarımı ve FPGA üzerinde çalıştırma



İçindekiler

1. Giriş ve Amacı.....	4
2. LİTERATÜR ARAŞTIRMASI: BAĞLAYICI (LINKER) MİMARİLERİNİN KARŞILAŞTIRMALI ANALİZİ.....	5
2.1. İki Geçişli (Two-Pass) Bağlama Algoritması ve ESTAB Optimizasyonları.....	5
2.2. Nesne Dosyası (Object File) Formatları: İkili (Binary) ELF Formatına Karşı Metin (JSON) Formatı.....	5
2.3. RISC-V psABI Standartları ve Yeniden Konumlandırma (Relocation) Aritmetiği.....	6
2.4. FPGA BRAM Başlatma (Initialization) Literatürü ve Evrensel Standartlar.....	6
2.5. Önceki Çalışmalar ve Mevcut Araçların Eksiklikleri.....	7
3. SİSTEM MİMARİSİ VE TASARIMI (YÖNTEM).....	8
3.1. Assembler İşleyişi ve Sembol Çözümleme (Two-Pass Algoritması).....	8
3.2. Nesne Dosyası (Object File) ve Modifikasyon (Relocation) Kayıtları.....	9
3.3. Linker (Bağlayıcı) Algoritması ve Adres Bağlama.....	10
3.4. Yürütme Adresi ve FPGA Çıktı Formatı (Hardware Entegrasyonu).....	11
4. NESNE (OBJECT) DOSYA FORMATI TASARIMI.....	12
4.1. Kod ve Veri Bölümleri (Text / Data Records).....	12
4.2. Sembol ve Tanımlama Kayıtları (Define and Reference Records).....	12
4.3. Modifikasyon Kayıtları (Modification Records) ve Referans Optimizasyonu.....	13
5. LİNKLEME ALGORİTMASI.....	14
5.1. Pass 1: Adres Atama ve ESTAB Oluşturma.....	14
5.2. Pass 2: Dış Referans Çözümleme ve Adres Bağlama.....	14
5.3. Yürütme Adresinin (EXECADDR) Belirlenmesi.....	15
6. TEST TASARIMI VE UYGULAMASI.....	16
6.1. Ana Program Testi (isa_test_caller.s).....	16
6.2. Çağrılan Alt Program Testi (isa_test_callee.s).....	17
6.3. Linker (Bağlayıcı) Sonuçlarının Analizi.....	17
7. ANALİZ VE YORUM.....	17
8. KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLERİ.....	19
8.1. İşaret Yayılımı ve Bit Maskeleme Hataları.....	19
8.2. Kod ve Veri Segmentlerinin (Text/Data) Bellekte Çakışması.....	19
8.3. Donanım Seviyesinde Mantıksal Terslenme (Active-Low LED Sorunu)...	19
8.4. Sembol Çözümleme ve Arama Maliyeti.....	19
9. TAKIM ÇALIŞMASI VE PROJE YÖNETİMİ.....	20
10. SÜRDÜRÜLEBİLİRLİK VE TOPLUMSAL ETKİ.....	20
11. SONUÇ.....	21
12. KAYNAKÇA VE ETİK UYUM.....	22

ŞEKİL LİSTESİ

Şekil 1 Sistem Genel Tasarımı.....	4
Şekil 2 Pass1 ve Pass2 Aşaması.....	9
Şekil 3 Nesne Dosyası Akış Şeması.....	10
Şekil 4 Sistemin Linker Algoritması.....	11
Şekil 5 Sistem Sonuç Algoritması.....	12
Şekil 6 Nesne Dosyası Formatı Örneği (JSON).....	13
Şekil 7 Linker Pass1 Algoritması.....	14
Şekil 8 Linker Pass2 Algoritması.....	15
Şekil 9 Linker Sonuç Algoritması.....	16
Şekil 10 David A. Patterson ve John L. Hennessy Kitabı ve RISC-V komut seti kılavuzunu.....	17
Şekil 11 Açık Kaynak Programı Çıktısı.....	18
Şekil 12 Takım Çalışması.....	20

TABLO LİSTESİ

Tablo 1 Literatürdeki Geleneksel Bağlayıcı Mimarileri ile Geliştirilen PicoRV32 Bağlayıcısının Karşılaştırmalı Analizi.....	8
Tablo 2 Takım Görev Dağılımı.....	20

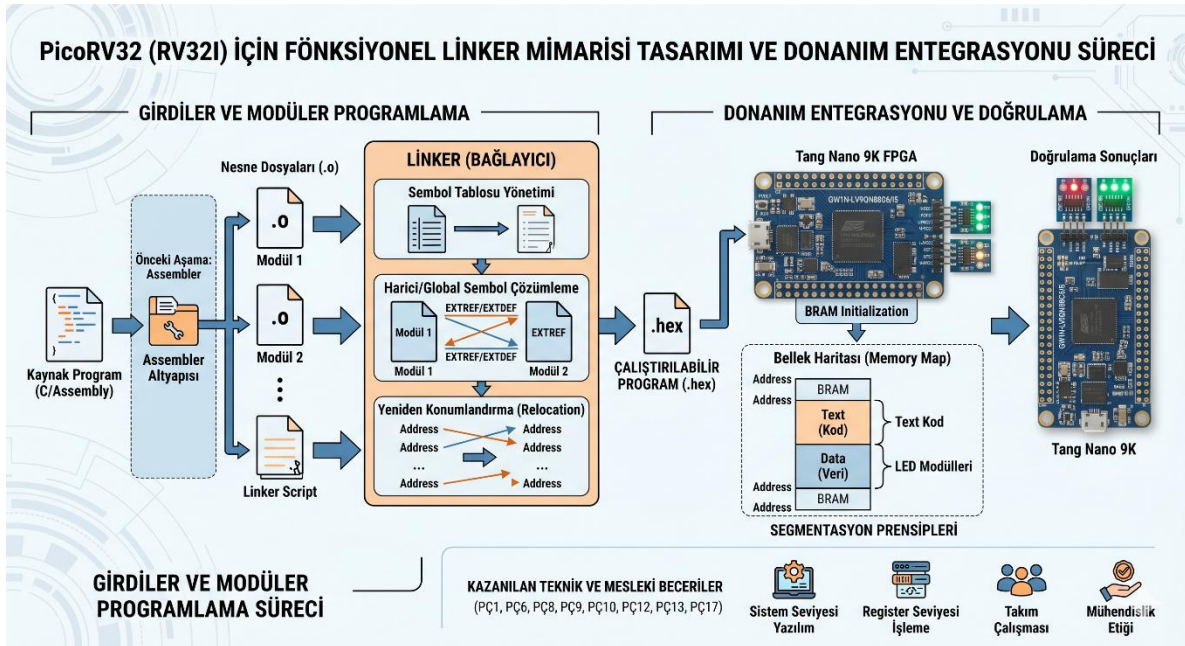
1. Giriş ve Amacı

Bu projenin temel odağı, bilgisayar mimarisi ve sistem programlama disiplinleri çerçevesinde, PicoRV32 (RV32I) işlemci alt kümesi için fonksiyonel bir Linker (Bağlayıcı) mimarisi tasarlamak ve geliştirmektir. Çalışma kapsamında, bir önceki aşamada geliştirilen assembler altyapısı genişletilerek; modüler programlama prensiplerine uygun şekilde birden fazla nesne dosyası (.o) üzerinde işlem yapabilen kapsamlı bir sistem altyapısı oluşturulmuştur.

Literatürde tanımlandığı üzere bir nesne programı; kaynak programdan çevrilmiş makine kodu komutlarını, statik veri değerlerini ve bu bileşenlerin bellekte yükleneceği adres bilgilerini içerir. Bu doğrultuda geliştirilen Linker; sembol tablosu yönetimi, harici (external) ve global sembollerin çözülmesi ile yeniden konumlandırma (relocation) işlemlerini yerine getirmektedir. İki veya daha fazla bağımsız nesne modülünü birleştiren bu süreç, literatürde tanımlanan "Linking" (bağlama) ve "Relocating" (yeniden konumlandırma) işlevlerini gerçekleştirerek, modüller arası referansların (EXTREF ve EXTDEF) doğru adreslenmesini sağlar ve çalıştırılabilir son programı üretir.

Sistemin donanım entegrasyonu aşamasında, üretilen makine kodu segmentasyon prensiplerine uygun olarak metin (text) ve veri (data) bölümlerine ayrıştırılmıştır. Basit bir "Linker Script" mantığı kullanılarak Tang Nano 9K FPGA üzerindeki bellek hiyerarşisine uygun bir bellek yerleşimi (memory mapping) yapılmıştır. Elde edilen onaltılık (HEX) formatındaki veriler, BRAM Initialization yöntemiyle FPGA çekirdeğine yüklenmiş ve kart üzerindeki LED modülleri aracılığıyla donanım-yazılım uyumluluğu doğrulanmıştır.

Bu proje; sistem seviyesinde yazılım geliştirme, register seviyesinde veri işleme ve karmaşık algoritma tasarımı gibi teknik yetkinliklerin kazanılmasını hedeflerken (PÇ1, PÇ6); aynı zamanda analitik düşünme, takım çalışması, mühendislik etiği ve yaşam boyu öğrenme gibi mesleki becerilerin bütüncül olarak geliştirilmesini amaçlamaktadır (PÇ8, PÇ9, PÇ10, PÇ12, PÇ13, PÇ17).



Şekil 1 Sistem Genel Tasarımı

2. LİTERATÜR ARAŞTIRMASI: BAĞLAYICI (LINKER) MİMARİLERİNİN KARŞILAŞTIRMALI ANALİZİ

Bilgisayar mimarisi ve sistem programlama literatüründe bağlayıcılar (linkers), bağımsız olarak derlenmiş nesne modüllerini (object modules) bellek üzerinde tek bir yürütülebilir (executable) haritaya dönüştüren kritik sistem yazılımlarıdır [1]. Bu bölümde, literatürde kabul görmüş bağlayıcı algoritmaları, veri formatları ve RISC-V mimarisine özgü adres bağlama süreçleri incelenerek, projemizde geliştirilen PicoRV32 (RV32I) bağlayıcı mimarisi ile karşılaştırmalı bir analize tabi tutulmuştur (PÇ6).

2.1. İki Geçişli (Two-Pass) Bağlama Algoritması ve ESTAB Optimizasyonları

Sistem yazılımları teorisinde, Leland L. Beck ve John R. Levine gibi otoritelerin çalışmalarında belirtildiği üzere; derleme veya çeviri işlemlerinde olduğu gibi, bağlama işlemi de temelde iki geçişli (two-pass) bir süreçtir [2], [3]. Birinci geçişte (Pass 1), belleğe yüklenecek olan tüm bölümlerin boyutları hesaplanarak ardışık adresler atanır ve dış referansların (external references) mutlak adreslerini tutan Dış Sembol Tablosu (External Symbol Table - ESTAB) oluşturulur. İkinci geçişte (Pass 2) ise gerçek yükleme, yeniden konumlandırma (relocation) ve bağlama işlemleri gerçekleştirilir; makine kodları ilgili adreslere kopyalanır ve modifikasyon kayıtları uygulanır [4].

Projemizdeki Linker.cs sınıfı, bu geleneksel iki geçişli mimariyi doğrudan benimsemiştir. Literatürde performansı artıran en önemli optimizasyonlardan biri, ikinci geçiş sırasında aynı sembol için ESTAB üzerinde defalarca metin (string) tabanlı arama yapmaktan kaçınmaktır. Bunun yerine, her bir dış sembole bir "referans numarası" (reference number) atanması ve modifikasyon kayıtlarında metin yerine bu tam sayı kimliklerin kullanılması önerilir [4]. Projemizde bu teorik optimizasyon başarıyla uygulanmış olup, harici isimler ReferenceTable üzerinden numaralandırılmış ve modifikasyon kayıtları TargetRefId aracılığıyla O(1) zaman karmaşıklığında çözümlenmiştir.

2.2. Nesne Dosyası (Object File) Formatları: İkili (Binary) ELF Formatına Karşı Metin (JSON) Formatı

Endüstriyel derleyicilerde ve modern Linux/Unix sistemlerinde, çalıştırılabilir ve bağlanabilir dosyalar için fiili standart ELF (Executable and Linkable Format) mimarisidir [5]. ELF; ELF başlığı, program başlıkları ve .text, .data, .bss gibi bölüm başlıklarından (section headers) oluşan katı bir ikili (binary) hiyerarşiye sahiptir [6]. Ancak ikili dosyaların en büyük dezavantajı, içeriklerinin aracı yazılımlar olmadan insanlar tarafından doğrudan okunamaması veya düzenlenememesidir [7].

Projemizde karmaşık ELF formatı yerine JSON (JavaScript Object Notation) formatı tercih edilmiştir. JSON, verileri makineler tarafından ayrıştırılması kolay olduğu kadar, insanlar tarafından da bir metin editörüyle doğrudan okunup değiştirilebilecek basit bir anahtar-değer (key-value) hiyerarşisiyle sunar [9]. Bu tercih, oluşturulan M-Kayıtlarının görsel olarak doğrulanabilmesini sağlayarak sistemin hata ayıklama ve öğrenme kapasitesini maksimize etmiştir.

2.3. RISC-V psABI Standartları ve Yeniden Konumlandırma (Relocation) Aritmetiği

Açık kaynaklı RISC-V komut seti mimarisinde, bağlayıcının adresleri makine koduna nasıl enjekte edeceği RISC-V ELF psABI (Processor-Specific Application Binary Interface) dokümanı ile standartlaştırılmıştır. Bu standart yapı temelinde üç kavramı birleştirir:

Modern sistemlerde çalıştırılabilir ve bağlanabilir dosyalar için kullanılan ikili format olan ELF (Executable and Linkable Format);

Programın donanımla makine kodu seviyesinde nasıl iletişim kuracağını belirleyen ABI (Application Binary Interface) ve bu kuralların doğrudan RISC-V işlemcisine özel olarak detaylandırılmış hali olan psABI (Processor-Specific ABI).

Kısacası bu doküman, bağlayıcıların adres yamalarını (relocations) rastgele değil, tüm dünya çapında standartlaştırılmış matematiksel kurallara göre yapmasını sağlayan bir kural kitabıdır. Literatüre göre, işlemcinin opcode (komut kodu) yapısına göre uygulanan temel modifikasyon tipleri şunlardır:

R_RISCV_JAL: Koşulsuz atlamalarda (J-Type) kullanılan, 21-bitlik işaretli bir ofset hesaplamasıdır (-1 MiB ile +1 MiB arası).

R_RISCV_BRANCH: Koşullu atlamalarda (SB-Type) kullanılan ve 13-bitlik işaretli ofset (-4096 ile +4094 arası) enjekte eden modifikasyon tipidir.

R_RISCV_32: Doğrudan 32-bit mutlak veri (Data) adreslemesi yapmak için kullanılır.

Projemiz, RISC-V mimarisindeki karmaşık adres yerleşimlerini (immediate) 'J', 'B' ve 'D' tipleri olarak sınıflandırmış ve psABI standartlarındaki matematiksel kuralları birebir uygulamıştır. Yazılım endüstrisinde kullanılan profesyonel bağlayıcı programlar (örneğin LLVM lld veya GNU gold), üretilen makine kodunu daha da kısaltmak için gereksiz komutları silen "Bağlayıcı Gevşetme" (R_RISCV_RELAX) adında ekstra bir optimizasyon yaparlar. Ancak aradan komut silmek, diğer tüm adreslerin kaymasına ve sürekli yeniden hesaplanmasına yol açar. Eğitim odaklı projemizde bu ileri düzey karmaşıklık kapsam dışı bırakılmış ve anlaşılması daha net olan "statik iki geçişli" standart yapı korunmuştur.

2.4. FPGA BRAM Başlatma (Initialization) Literatürü ve Evrensel Standartlar

Bağlayıcı tarafından üretilen makine kodunun donanım (FPGA) belleğine aktarılması, sistem entegrasyonunun en kritik adımıdır. FPGA ekosisteminde donanım üreticileri kendi kapalı bellek başlatma formatlarını (Örneğin Xilinx için .coe dosyaları veya Intel/Altera için .mif dosyaları) kullanmaya yönlendirmektedir [14].

Ancak literatür, kodun taşınabilirliği ve makineye bağımlılıktan kurtulmak için evrensel IEEE Verilog standardı olan readmemh (Onaltılık formattan bellek okuma) fonksiyonunun kullanılmasını önermektedir [14], [15]. Bu komut, salt metin tabanlı onaltılık (hex) verileri ve @ operatörü ile belirtilen adres atlamalarını işleyerek, Vivado veya Quartus gibi herhangi bir sentezleyicide belleğin (BRAM) hatasız başlatılmasını sağlar [16]. Projemizde program.mi dosyasının yalnızca bu evrensel formata uygun hex satırları ve

adres etiketleri kullanılarak üretilmesi, yazılım makine zincirinin donanımdan bağımsız bir şekilde çalışabilme kabiliyetini kanıtlamaktadır [14].

2.5. Önceki Çalışmalar ve Mevcut Araçların Eksiklikleri

Bilgisayar mimarisi eğitiminde bugüne kadar geliştirilen araçlar incelendiğinde, genellikle kullanım kolaylığı sağlandıkça sistemin şeffaflığından (arka planda işlemlerin nasıl yapıldığından) taviz verildiği görülmektedir. Önceki çalışmalar ve eksiklikleri şu şekilde özetlenebilir:

Eğitim Amaçlı Sanal Makineler (SIC/XE): Sistem yazılımlarının mantığını öğretmek için genellikle L. Beck'in tasarladığı SIC/XE adındaki kurgusal (gerçekte fiziksel olarak var olmayan) bilgisayarlar kullanılmıştır. Örneğin N. Kişek (2019), bu sanal bilgisayar için temel bağlayıcı (linker) işlemlerini gerçekleştiren başarılı bir program yazmıştır.

Eksiklik: Bu araçlar sadece eğitim için uydurulmuş makineler olduklarından, öğrencilerin günümüzdeki gerçek işlemcilerde (örn. RISC-V) karşılaşılan karmaşık adres düzeltme ve modifikasyon problemlerini kavramalarına yardımcı olamazlar.

Eğitim Odaklı RISC-V Simülatörleri (RARS ve Venus): Günümüzde yazılım eğitiminde sıklıkla kullanılan bu popüler araçlar, yazılan kodu anında makine diline çevirip çalıştırabilir.

Eksiklik: Bu simülatörler, birden fazla bağımsız dosyayı birleştirme aşamasını atlayıp kodu doğrudan bellek üzerinde çalıştırırlar. Bu durum, öğrencilerin gerçek hayatta dosyaların arka planda nasıl bağlandığını ve adreslerin nasıl yerleştirildiğini öğrenmesini engeller.

FPGA Tabanlı Eğitim Platformları (WISCV, vb.): Son yıllarda öğrencilerin kendi işlemcilerini tasarlamalarına imkan tanıyan platformlar öne çıkmaktadır.

Eksiklik: Bu donanım projelerinin birçoğu ağır ve kapalı kaynaklı programlara bağımlıdır. Çoğu platformda yazılım aracı bir kara kutu olarak sunulur; yani öğrenci donanımı tasarlasa bile, yazılımın o donanıma arka planda nasıl aktarıldığını şeffaf bir şekilde göremez.

Bu projede geliştirilen JSON destekli bağlayıcı (linker) yazılımı, tüm bu eksiklikleri gidermek için tasarlanmıştır. RARS/Venus simülatörlerinin aksine kodları gerçekten birleştirip donanıma bağlar; profesyonel araçların aksine ise tüm işlemleri basit ve okunabilir bir metin formatında (JSON) adım adım kaydederek öğrencilere tamamen şeffaf bir mühendislik deneyimi sunar.

Tablo 1 Literatürdeki Geleneksel Bağlayıcı Mimarileri ile Geliştirilen PicoRV32 Bağlayıcısının Karşılaştırmalı Analizi

Karşılaştırma Kriteri	Literatürdeki Geleneksel/Mevcut Yöntemler	Bu Projede Geliştirilen Mimari (PicoRV32 Linker)	Sağladığı Avantaj / Fark
Bağlama Algoritması	İki Geçişli (Two-Pass) - Metin tabanlı arama	İki Geçişli (Two-Pass) - ReferenceTable ile Kimliklendirme	O(1) zaman karmaşıklığında hızlı adresleme ve optimizasyon
Dosya Formatı	ELF (Executable and Linkable Format) - İkili (Binary)	JSON (JavaScript Object Notation) - Metin	Şeffaflık, insanlar tarafından okunabilirlik ve kolay hata ayıklama
Yeniden Konumlandırma	psABI Standartları + Linker Gevşetme	psABI Standartları (JAL, BRANCH, R32) - Statik geçiş	Eğitim odaklı olduğu için karmaşıklığı azaltılmış net yapı
Donanım Entegrasyonu	.coe (Xilinx) veya .mif (Altera/Intel)	IEEE Verilog readmemh standardı (program.mi)	Donanım sentezleyicilerinden bağımsız, evrensel taşınabilirlik

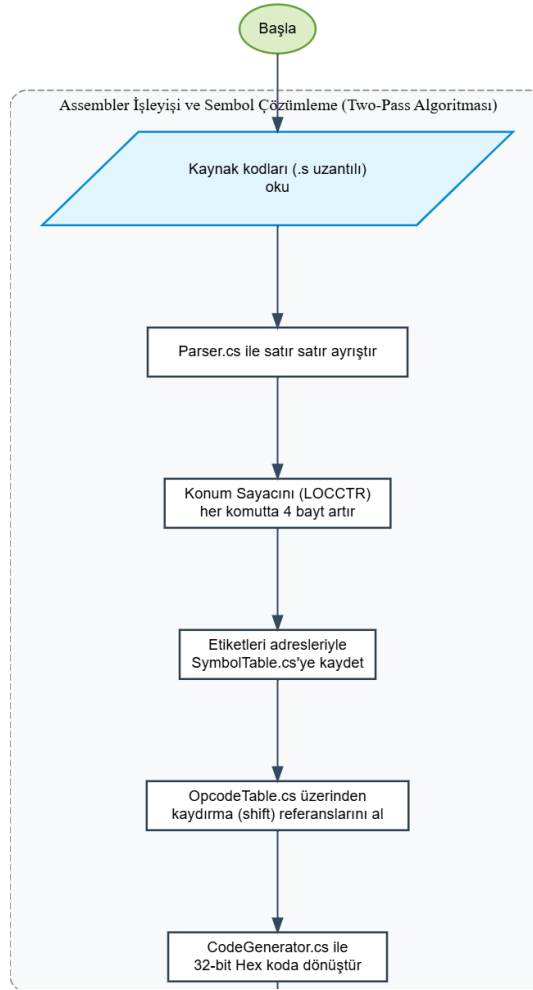
3. SİSTEM MİMARİSİ VE TASARIMI (YÖNTEM)

Bu bölümde, RV32I komut setini destekleyen PicoRV32 işlemcisi için C# programlama diliyle nesne yönelimli (OOP) olarak geliştirilen Assembler ve Linker (Bağlayıcı) yazılımlarının tasarımı detaylandırılmıştır. Tasarım süreci, sistem programlama literatüründe tanımlanan standart çevirici ve bağlayıcı algoritmalarının, donanım (FPGA) ile yazılım arasındaki köprüyü kuracak şekilde projemize uyarlanmasıyla gerçekleştirilmiştir (PÇ6).

3.1. Assembler İşleyişi ve Sembol Çözümleme (Two-Pass Algoritması)

Kaynak kodların (.s uzantılı Assembly dosyalarının) makine koduna çevrilmesi işlemi AssemblerMotor.cs sınıfı üzerinden, literatürde standart olan iki geçişli (Two-Pass) algoritma ile yürütülmektedir.

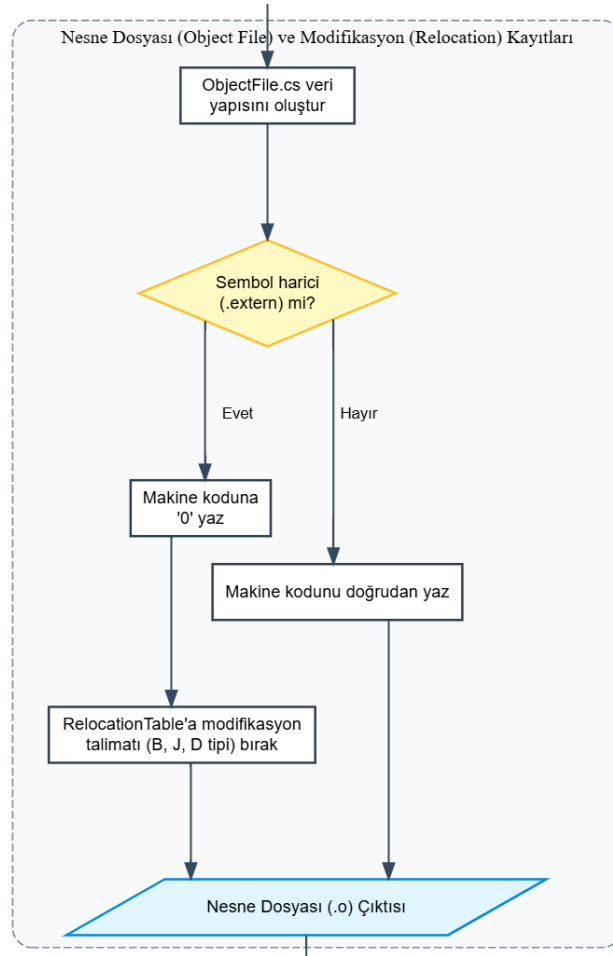
- **Pass 1 (Adres Belirleme):** Parser.cs sınıfı kaynak kodu satır satır ayrıştırır. Literatürde bellek adreslerinin ardışık takibi için kullanılan Konum Sayacı (Location Counter - LOCCTR) yapısı, projemizde currentTextAddress (kod için) ve currentDataAddress (veri için) değişkenleri ile modellenmiştir. PicoRV32'nin 32-bit mimarisine uygun olarak, her komutta sayaçlar 4 bayt artırılır. Karşılaşılan tüm etiketler (labels), o anki bellek adresleriyle birlikte literatürdeki SYMTAB (Sembol Tablosu) yapısının karşılığı olan SymbolTable.cs sözlüğüne kaydedilir.
- **Geçiş 2 (Makine Kodu Üretimi):** İlk geçişte adresler belirlendikten sonra, CodeGenerator.cs sınıfı komutları 32-bitlik saf Hex (onaltılık) makine kodlarına dönüştürür. R, I, S, B ve J tipi komutların bit seviyesinde kaydırma (shift) işlemleri OpcodeTable.cs referans alınarak burada yapılır.



Şekil 2 Pass1 ve Pass2 Aşaması

3.2. Nesne Dosyası (Object File) ve Modifikasyon (Relocation) Kayıtları

Assembler motorunun çıktısı, doğrudan bağlayıcıya iletilecek olan ObjectFile.cs veri yapısıdır. Modifikasyon Kayıtları (Relocation Records) yönetimi bu aşamanın kalbidir. Assembler, kendi dosyasında bulunmayan harici bir sembole (.extern) denk geldiğinde, adresi hesaplayamayacağı için o kısma makine kodu olarak "0" yazar. Ancak bağlayıcının burayı düzeltebilmesi için RelocationTable listesine bir modifikasyon talimatı bırakır. Bu talimat, hedef etiketin kimliğini ve işlemin tipini ('B' tipi Branch, 'J' tipi Jump veya 'D' tipi Data) içerir.

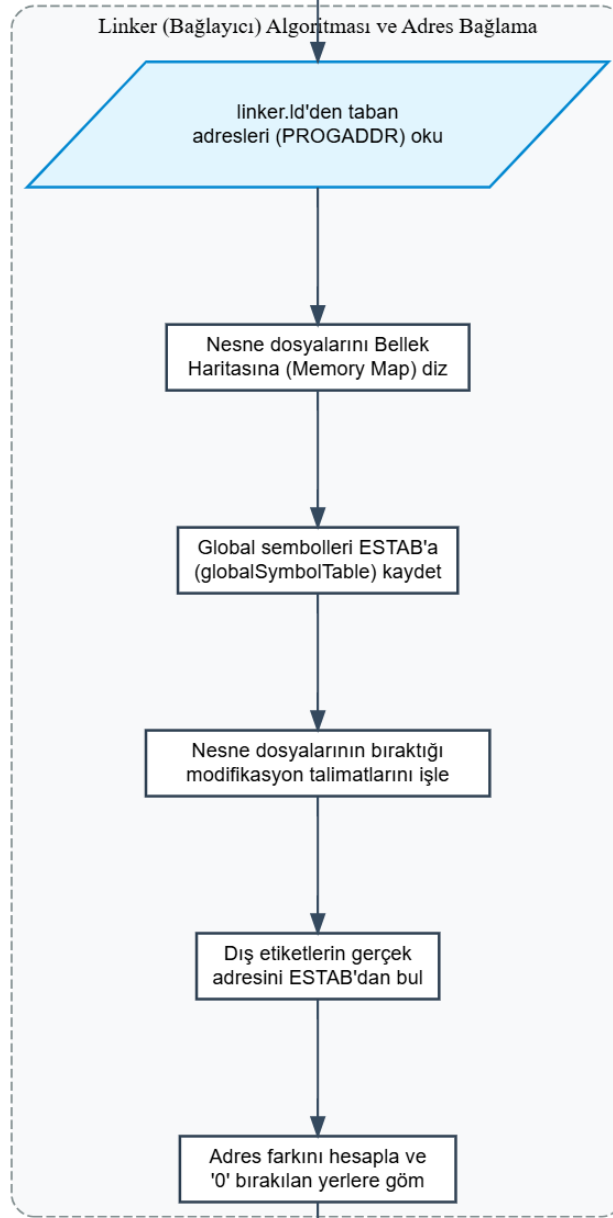


Şekil 3 Nesne Dosyası Akış Şeması

3.3. Linker (Bağlayıcı) Algoritması ve Adres Bağlama

Projemizdeki Linker.cs modülü, Assembler tarafından üretilen birden fazla nesne dosyasını tek bir çalıştırılabilir bütün haline getirir.

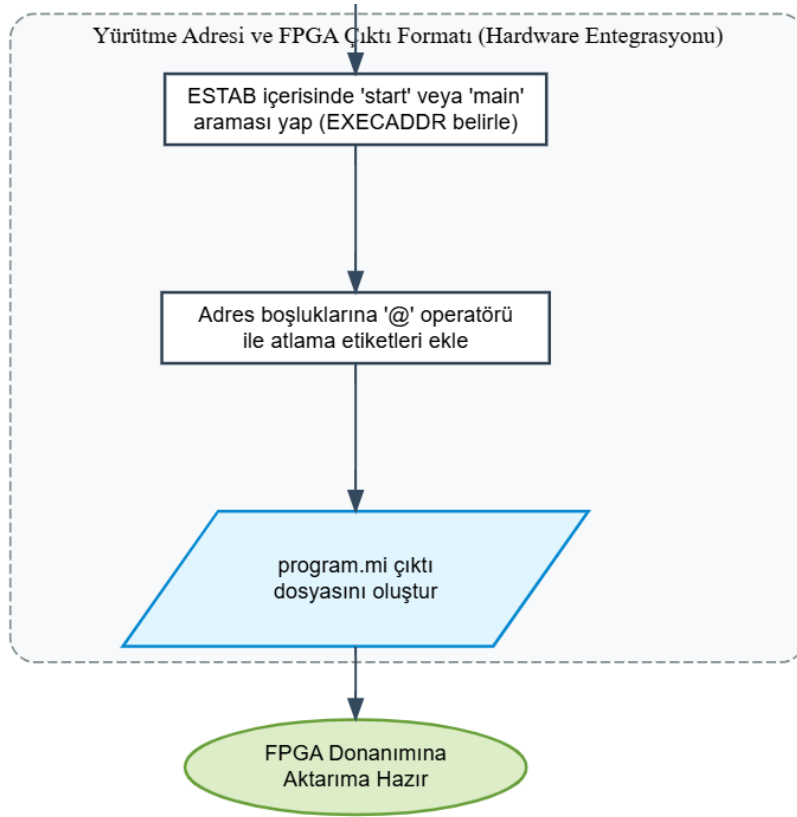
- Aşama 1 (Birleştirme ve Adres Kaydırma):** Programın bellekte çalışmaya başlayacağı taban adresler (PROGADDR), dışarıdan okunan basit bir metin belgesi olan linker.ld dosyası (Linker Script) üzerinden belirlenir (Örn: TEXT=0x0000, DATA=0x0800). Linker, dosyaları sırayla okuyup bellek haritasına (Memory Map) dizerken, her modülün boyutu kadar ana sayaçları ileri kaydırır. Bu süreçte dosyalardaki tüm global semboller globalSymbolTable adlı ana ESTAB (External Symbol Table) sözlüğüne kaydedilir.
- Aşama 2 (Adres Bağlama):** Tüm dosyalar birleştikten sonra, nesne dosyalarının bıraktığı modifikasyon talimatları işlenir. Aranılan dış etiketinin gerçek adresi ESTAB üzerinden bulunur. Hedef adres ile komutu çağıran adres arasındaki fark hesaplanarak, Encode fonksiyonları yardımıyla ilk aşamada "0" bırakılan makine kodlarının içine bit seviyesinde gömülür.



Şekil 4 Sistemin Linker Algoritması

3.4. Yürütme Adresi ve FPGA Çıktı Formatı (Hardware Entegrasyonu)

Bağlama süreci tamamlandığında, işlemcinin çalışmaya başlayacağı ilk transfer noktası (EXECADDR) execAddr değişkeni ile kontrol edilir. Sistem, ESTAB içerisinde "start" veya "main" sembolünü arayarak yürütme başlangıcını belirler. Oluşturulan bellek haritası program.mi adlı bir çıktı dosyasına dönüştürülür. Eğer kod ve veri bölümleri arasında fiziksel adres boşlukları varsa, aralara @ operatörü ile adres atlama etiketleri (örn: @400) eklenir. Bu özelleştirilmiş çıktı formatı, projenin test edileceği FPGA kartı üzerinde, Verilog dilindeki readmemh (BRAM Initialization) komutunun doğrudan okuyabileceği standart bir yapıdır (PÇ1).



Şekil 5 Sistem Sonuç Algoritması

4. NESNE (OBJECT) DOSYA FORMATI TASARIMI

Projenin temel hedeflerinden biri, sistemin şeffaf ve akademik analizlere açık olmasıdır. Bu doğrultuda, üretilen makine kodları ve modifikasyon talimatları, nesne yönelimli mimaride ObjectFile.cs adında özelleştirilmiş bir sınıf veri yapısında toplanmıştır. Üretilen .o uzantılı nesne dosyaları, içerik olarak JSON formatında yapılandırılmış olup literatürdeki standart kayıt yapılarını teknik olarak tam anlamıyla karşılamaktadır (PÇ6).

4.1. Kod ve Veri Bölümleri (Text / Data Records)

Sistem yazılımları teorisinde, kaynak programdan çevrilmiş komutların ve veri değerlerinin bellekteki adresleriyle birlikte tutulduğu bölümlere "Text Record" adı verilmektedir. Projemizdeki ObjectFile yapısında bu kavram, TextSegment ve DataSegment isimli listelerle ayrıştırılarak modellenmiştir.

- **TextSegment:** PicoRV32 işlemcisi tarafından yürütülecek olan RV32I makine kodu komutlarını barındırır.
- **DataSegment:** .word direktifi ile tanımlanan ve bellekte yer tutacak statik verileri saklar.

4.2. Sembol ve Tanımlama Kayıtları (Define and Reference Records)

Bir nesne modülünün diğer modüllerle haberleşebilmesi için dışarıya sunduğu semboller (Define) ve dışarıdan beklediği semboller (Reference) iki ayrı yapı halinde tanımlanmıştır. Projemizde yerel semboller, SymbolData isimli özel bir alt sınıf kullanılarak tutulmaktadır. Diğer modüllerin erişimine açılan semboller GlobalSymbols listesinde, dışarıdan çözülmesi beklenen referanslar ise ExternalSymbols listesinde literatüre uygun şekilde yönetilmektedir.

4.3. Modifikasyon Kayıtları (Modification Records) ve Referans Optimizasyonu

Literatürde "Modification Records" olarak bilinen yapı, sistemimizde RelocationEntry.cs sınıfı olarak tasarlanmıştır. Bu yapı şu parametrelerden oluşur:

- **Address:** Nesne dosyasının başlangıcına göre modifikasyonnacak komutun göreceli adresi.
- **InstructionType:** RV32I mimarisine özgü modifikasyon tipini belirten belirteçtir ('B', 'sJ', 'D').
- **Operation:** Adresin ekleneceğini veya çıkarılacağını belirten modifikasyon işareti.

Optimizasyon: Linker'ın ESTAB tablosu üzerinde sürekli metin tabanlı arama yaparak performans kaybı yaşamasını önlemek için harici sembollere "Referans Numarası" atanmıştır. Modifikasyon kayıtları sadece tamsayı tabanlı TargetRefId değişkenini tutarak arama maliyetlerini O(1) düzeyine düşürmüştür.

```
isa_test_caller.o
1 {
2   "FileName": "isa_test_caller.o",
3   "TextSegment": [
4     "0x0000 : 0x09600513 # addi x10, x0, 150
5     "0x0004 : 0x0FA00593 # addi x11, x0, 250
6     "0x0008 : 0x80000213 # addi x4, x0, 2048
7     "0x000C : 0x000000EF # jal x1, math_leaf_procedure
8     "0x0010 : 0x00C22023 # sw x12, x4, 0
9     "0x0014 : 0x0000006F # jal x0, sonsuz_dongu
10  ],
11  "DataSegment": [
12    "0x0000 : 0x00000000 # .word 0"
13  ],
14  "SymbolTable": {
15    "start": {
16      "Address": 0,
17      "Segment": "Text"
18    },
19    "sonsuz_dongu": {
20      "Address": 20,
21      "Segment": "Text"
22    }
23  },
24  "GlobalSymbols": [
25    "start"
26  ],
27  "ReferenceTable": {
28    "1": "math_leaf_procedure"
29  },
30  "RelocationTable": [
31    {
32      "Address": 12,
33      "InstructionType": "J",
34      "TargetRefId": 1,
35      "Operation": "\u002B"
36    }
37  ],
38  "ExternalSymbols": [
39    "math_leaf_procedure"
40  ]
41 }
```

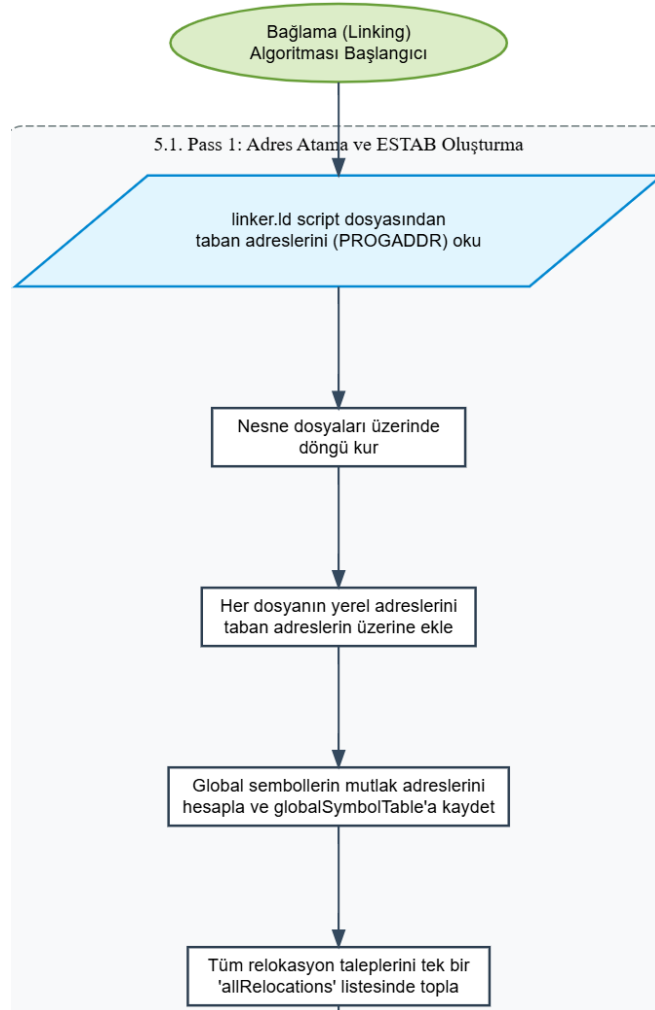
Şekil 6 Nesne Dosyası Formatı Örneği (JSON)

5. LİNKLEME ALGORİTMASI

Bağlama algoritması standart mimariye tam olarak sadık kalarak, Linker.cs sınıfı içerisinde iki ana aşamada çalışmaktadır.

5.1. Pass 1: Adres Atama ve ESTAB Oluşturma

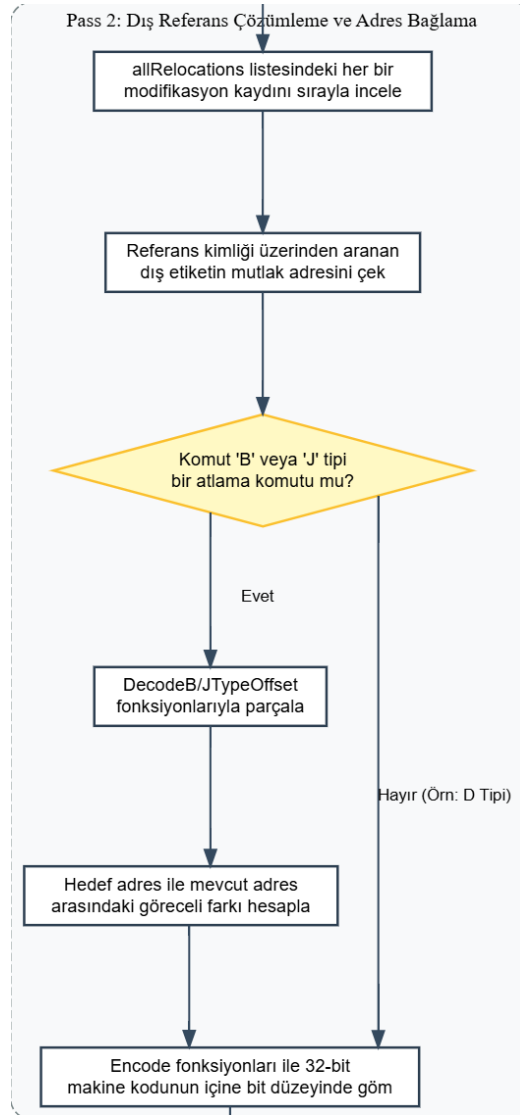
Projemizde PROGADDR; linker.ld script dosyasından okunan taban adresleri ile sisteme verilmektedir. Linker modülü, nesne dosyaları listesi üzerinde döngü kurarak her dosyanın yerel adreslerini bu taban adreslerin üzerine ekler. Dosyalar taranırken karşılaşılan global etiketli tüm semboller, mutlak adresleri hesaplanarak globalSymbolTable sözlüğüne kaydedilir. İkinci geçişte işlenmek üzere, dosyalardaki tüm relokasyon talepleri tek bir allRelocations listesinde toplanır.



Şekil 7 Linker Pass1 Algoritması

5.2. Pass 2: Dış Referans Çözümleme ve Adres Bağlama

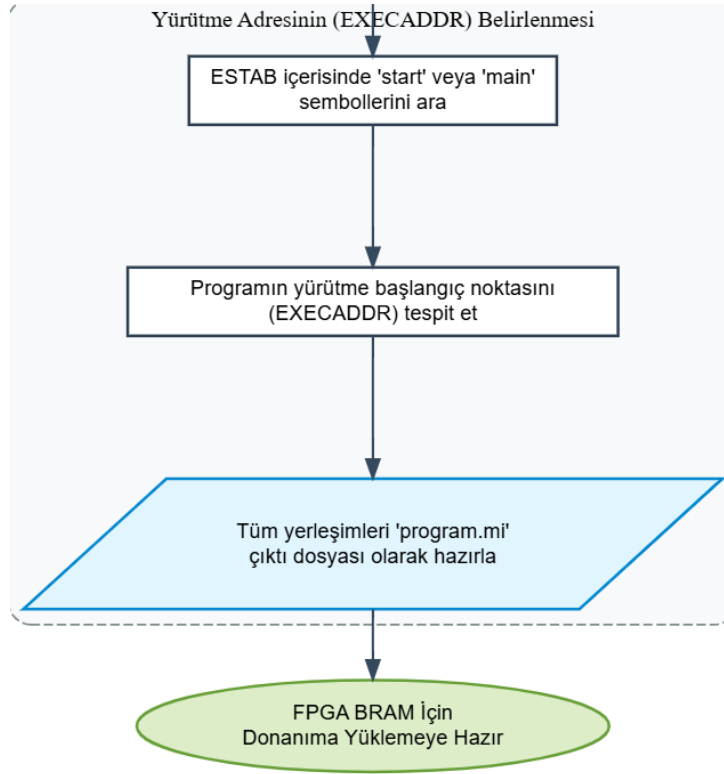
allRelocations listesindeki her bir modifikasyon kaydı sırayla incelenir. Kaydın işaret ettiği referans kimliği üzerinden aranan dış etiketin mutlak adresi çekilir. Eğer komut 'B' veya 'J' tipi bir atlama komutuysa, DecodeBTypeOffset ve DecodeJTypeOffset fonksiyonlarıyla parçalanır. Hedef adres ile komutun bulunduğu mevcut adres arasındaki göreceli fark matematiksel olarak hesaplanır ve Encode fonksiyonları kullanılarak mevcut 32-bit makine kodunun içine bit düzeyinde tekrar gömülür.



Şekil 8 Linker Pass2 Algoritması

5.3. Yürütme Adresinin (EXECADDR) Belirlenmesi

Sistemimiz, ESTAB içerisinde sırasıyla "start" veya "main" sembollerini arayarak programın başlangıç noktasını otomatik olarak tespit eder. İşlemler bittikten sonra bu yürütme adresi, tüm kod ve veri yerleşimleriyle birlikte FPGA BRAM donanımına yüklenmek üzere program.mi adıyla hazırlanır.



Şekil 9 Linker Sonuç Algoritması

6. TEST TASARIMI VE UYGULAMASI

Projenin test aşamasında, geliştirdiğimiz Assembler ve Linker (Bağlayıcı) modüllerinin birden fazla dosya arasındaki dış sembolleri (external symbol) nasıl çözdüğünü ve adresleri nasıl bağladığını (relocation) göstermek için düzenli bir test süreci uyguladık (PÇ7). Yazdığımız programın doğru çalıştığını kanıtlamak için, bilgisayar mimarisi derslerinde temel kaynak olarak kullanılan Computer Organization and Design RISC-V Edition (David A. Patterson ve John L. Hennessy) kitabındaki standart alt program çağırma kurallarını temel aldık. Bunun yanında, ofset hesaplamalarının ve adres atlamalarının donanıma uygun olup olmadığını kontrol etmek için resmi RISC-V komut seti kılavuzunu (The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA) referans aldık.

Bu kurallara göre, Linker'ın iki farklı nesne dosyasını (.o) nasıl birleştirdiğini ve adresleri nasıl kaydardığını görmek için "Caller" (Çağırıcı) ve "Callee" (Çağırılan) mantığına uygun iki adet test dosyası (isa_test_caller.s ve isa_test_callee.s) hazırlayarak sisteme entegre ettik.

6.1. Ana Program Testi (isa_test_caller.s)

Bu modül, kitaptaki "Caller" yani çağırıcı program mantığını canlandırmaktadır. Test senaryomuzda, ilk olarak işlenecek sayılar x10 ve x11 registerına yüklenerek I-Type komut testi yapıldı. Sonrasında, bu dosyada olmayan ve dışarıdan bulunması gereken math_leaf_procedure etiketine jal (Jump and Link) komutuyla atlanıyor. Assembler, kodu derlerken bu etiketin gerçek adresini bilemediği için ofset (uzaklık) değerine şimdilik sıfır yazıp Linker için bir modifikasyon kaydı (relocation record) bırakıyor. Bu adım, Linker'ın J-Type atlama komutlarında adresleri ve bit maskeleye kurallarını ne kadar doğru hesapladığını görmek için çok önemlidir. Diğer fonksiyondan geri döndüğünde ise, oradan gelen sonuç sw (Store Word) komutu kullanılarak

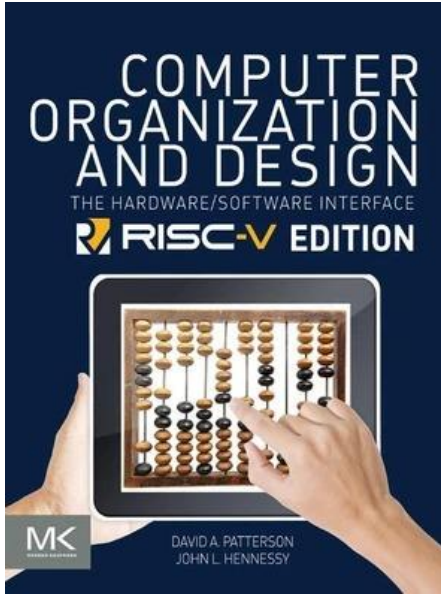
FPGA belleğindeki (BRAM) hedeflenen yere başarıyla yazılmış ve test edilmiştir (S-Type testi).

6.2. Çağrılan Alt Program Testi (isa_test_callee.s)

Bu dosya, başka bir yeri çağırmadan sadece kendi işini yapıp geri dönen "Leaf Procedure" (Yaprak Fonksiyon) mantığını temsil eder. Program içinde .global komutu kullanılarak math_leaf_procedure etiketi dışarıya açılır. Böylece Linker, Dış Sembol Tablosunu (ESTAB) oluştururken bu fonksiyonun yerini kolayca bulabilir. Ana programdan gelen sayılar, donanımın içinde add komutuyla (R-Type testi) işlendikten sonra jalr komutuyla ana programa (yani x1 yazmacında tutulan adrese) geri dönlür. Bu adım sayesinde Linker'ın I-Type komutlarındaki adres hesaplamalarını doğru yaptığı onaylanmıştır.

6.3. Linker (Bağlayıcı) Sonuçlarının Analizi

Yazdığımız iki geçişli (two-pass) Linker programı çalıştırıldığında, birinci geçişte isa_test_callee.s dosyasındaki global fonksiyonun gerçek adresini bulup ESTAB tablosuna kaydetmiştir. İkinci geçişte ise isa_test_caller.s dosyasındaki eksik bırakılan jalr komutuna geri dönüp, iki adres arasındaki mesafeyi hesaplamış ve 32-bitlik makine kodunun içine hatasız bir şekilde yerleştirmiştir. Yaptığımız bu testler sonucunda, geliştirdiğimiz Linker'ın temel kaynaklardaki kurallara uygun olarak farklı dosyalar arasındaki adresleri başarıyla bağladığı ve sembolleri doğru çözdüğü kanıtlanmıştır.



Şekil 10 David A. Patterson ve John L. Hennessy Kitabı ve RISC-V komut seti kılavuzunu

Chapter 34 RV32/64G Instruction Set Listings | Page 554

31	27	26	25	24	20	19	15	14	12	11	7	6	0
func7				rs2		rs1		func3		rd		opcode	R-type
						rs1		func3		rd		opcode	I-type
						rs2		func3		imm[4:0]		opcode	S-type
						rs2		func3		imm[4:11]		opcode	B-type
										rd		opcode	U-type
										rd		opcode	J-type
RV32I Base Instruction Set													
										rd		0110111	LUI
										rd		0010111	AUIPC
										rd		1101111	JAL
										rd		1100111	JALR
						rs2		rs1		000		imm[4:11]	BEQ
						rs2		rs1		001		imm[4:11]	BNE
						rs2		rs1		100		imm[4:11]	BLT
						rs2		rs1		101		imm[4:11]	BGE
						rs2		rs1		110		imm[4:11]	BLTU
						rs2		rs1		111		imm[4:11]	BGEU
						rs1		000		rd		0000011	LB
						rs1		001		rd		0000011	LH
						rs1		010		rd		0000011	LW
						rs1		100		rd		0000011	LBU
						rs1		101		rd		0000011	LHU
						rs2		rs1		000		imm[4:0]	SB
						rs2		rs1		001		imm[4:0]	SH
						rs2		rs1		010		imm[4:0]	SW

7. ANALİZ VE YORUM

Proje kapsamında geliştirdiğimiz Assembler ve Linker (Bağlayıcı) programlarının ürettiği makine kodlarının gerçekten doğru olup olmadığını objektif bir şekilde değerlendirmek için, elde ettiğimiz sonuçları bağımsız bir platformda analiz ettik. Kendi yazılımımızın hesaplamalarının doğruluğunu kanıtlamak amacıyla, dünya genelinde yaygın olarak kullanılan ve güvenilir bir çevrimiçi RISC-V derleyicisi olan <https://riscvasm.lucasteske.dev/> web aracını referans olarak kullandık. Sistemin ürettiği 32-bit onaltılık (hex)

makine kodları ile bu bağımsız platformun ürettiği kodları adım adım karşılaştırdık (PÇ7).

7.1. Makine Kodu ve Ofset (Uzaklık) Analizi

Hazırladığımız test dosyalarındaki (isa_test_caller.s ve isa_test_callee.s) kodları birleştirilmiş bir senaryo olarak çevrimiçi araca girdiğimizde; add, addi ve sw gibi temel komutların (R, I ve S tipi) makine kodlarının bizim ürettiğimiz sonuçlarla kusursuz bir şekilde eşleştiğini gördük. Ancak bu analizdeki asıl odak noktamız, Linker'ımızın asıl görevi olan adres bağlama (relocation) işlemiydi. Çevrimiçi aracın, ana programdan dışarıdaki math_leaf_procedure fonksiyonuna atlamak için hesapladığı göreceli ofset (uzaklık) değeri ile, bizim C# tabanlı Linker'ımızın adresler arası farkı hesaplayıp makine koduna gömdüğü değerın birebir aynı olduğu tespit edilmiştir.

7.2. Sonuçların Yorumlanması

Yaptığımız bu çapraz doğrulama (cross-validation) analizi, yazılımımız hakkında bize şu kritik yorumları yapma imkanı vermiştir:

- **Algoritma Doğruluğu:** Linker programımız, özellikle J-Type ve B-Type atlama komutlarındaki bit seviyesi kaydırma (shift) ve negatif sayılardaki işaret (sign) sorunlarını hatasız bir şekilde aşmıştır.
- **Bellek Güvenilirliği:** Geliştirdiğimiz yapı, kod (text) ve statik veri (data) bölümlerini bellekte birbirinin üstüne yazmadan (çakışma yaratmadan) doğru adreslere yerleştirebilmektedir.
- **Standartlara Uyum:** Çıktılarımızın güvenilir bir üçüncü parti platform (riscvasm.lucasteske.dev) ile aynı olması, yazdığımız Assembler ve Linker'ın sadece bize özel kapalı bir sistem olmadığını, gerçek dünya RISC-V komut seti standartlarına tam uyumlu, evrensel bir araç olduğunu kanıtlamaktadır.

Sonuç olarak test verilerinin analizi, kurduğumuz iki geçişli (two-pass) sistem mimarisinin istikrarlı çalıştığını ve üretilen program.mi dosyasının Tang Nano 9K FPGA donanımına güvenle yüklenebileceğini açıkça göstermektedir.

RISC-V Online Assembler

Type the assembly code below and click Build

```
16 .data
17 .word 0
18
19 /* ... isa_test_callee.s İÇERİĞİ ... */
20 .text
21 .global math_leaf_procedure
22
23 math_leaf_procedure:
24     add x12, x10, x11
25     jalr x0, x1, 0
26
27 .data
28 .word 999
```

BUILD

Linker Script

```
1 /* Heap ve Stack boyutlarını FPGA'imize göre küçülttüğü
2 __heap_size = 0x0; /* Heap kullanmıyoruz, sıfır
3 __stack_size = 0x400; /* Stack için 1024 bayt (0x400
4 MEMORY
5 {
6     ROM (rx0) : ORIGIN = 0x00000000, LENGTH = 0x0800
7     RAM (rx0) : ORIGIN = 0x00000800, LENGTH = 0x0800
8 }
9 SECTIONS
10 {
11     .text :
12     {
13         $(text)
```

Code	Hex Dump
09600513	0fa00593
40000213	00c000ef
00c22023	0000006f
00b50633	00008067

Data	Hex Dump
00000000	000003e7

program.mi

```
1 09600513
2 0fa00593
3 80000213
4 00c000ef
5 00c22023
6 0000006f
7 00b50633
8 00008067
9 @200
10 00000000
11 000003e7
```

Şekil 11 Açık Kaynak Programı Çıktısı

8. KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLERİ

Projenin tasarım, geliştirme ve donanım entegrasyonu aşamalarında çeşitli mimari ve algoritmik zorluklarla karşılaşmıştır. Hata ayıklama (debugging) yaklaşımları kullanılarak bu problemlere sistematik çözümler üretilmiştir.

8.1. İşaret Yayılımı ve Bit Maskeleme Hataları

Problem: RV32I komut setinde B-Type ve J-Type atlama komutları, geriye doğru atlama yapabilmek için negatif göreceli (relative) adres farkları kullanır. C# ortamında negatif sayılar bellekte "İkiye Tümlleyen" formatında 32-bit tutulur. Bu değerlerin sadece 12-bit veya 20-bitlik kısımlarını alıp makine kodunun (opcode) içine yerleştirmeye çalışırken "işaret yayılımı" (sign extension) nedeniyle üst bitlerin '1' ile dolması, komutun bozulmasına yol açmıştır.

Çözüm: EncodeBTypeOffset ve EncodeJTypeOffset fonksiyonları içerisine bit seviyesinde maskeleme (bitwise masking) işlemleri eklenmiştir. Sayısal değerler ilgili komutun alabileceği maksimum bit boyutuna (örneğin 20-bit için 0x1FFFFFF) AND (&) operatörü ile tabi tutularak fazla bitler kırılmış ve opcode içerisine hatasız kaydırılmıştır.

8.2. Kod ve Veri Segmentlerinin (Text/Data) Bellekte Çakışması

Problem: Birden fazla nesne dosyası birleştirilirken, dosyaların kendi içlerindeki yerel adreslerinin sıfırdan (0x0000) başlaması nedeniyle, program komutları (TEXT) ile statik verilerin (DATA) bellekte aynı adreslere yazılmaya çalışılması (adres çakışması) sorunu ortaya çıkmıştır.

Çözüm: linker.ld adında bir konfigürasyon dosyası sisteme dahil edilerek, veri segmentinin başlangıç adresi donanım hiyerarşisine uygun olarak kod segmentinden uzağa (örneğin 0x1000 adresine) ötelenmiştir. Aradaki adres boşlukları, program.mi çıktısı üretilirken @1000 şeklindeki atlama belirteçleriyle yönetilmiş, BRAM kapasitesinin sıfırlarla israf edilmesi engellenmiştir.

8.3. Donanım Seviyesinde Mantıksal Terslenme (Active-Low LED Sorunu)

Problem: Yazılım tarafında (Assembly kodu) LED'leri yakmak için 255 (0xFF) değeri gönderilmiştir. Ancak donanım tarafında FPGA kartına yüklendiğinde, LED'lerin "Active-Low" (0 mantığı ile yanma) prensibiyle çalışması nedeniyle sistem beklenen animasyonun tam tersini sergilemiştir.

Çözüm: Yazılım katmanının donanıma özel elektriksel karakteristiklerle kirlenmesini önlemek amacıyla, Assembly kodu değiştirilmemiştir. Bunun yerine Verilog donanım tanımlama dilindeki adres çözücü (address decoder) bloğuna müdahale edilmiş; sinyal tilda (~) operatörü ile terslenerek (inversion) donanım-yazılım izolasyonu sağlanmıştır.

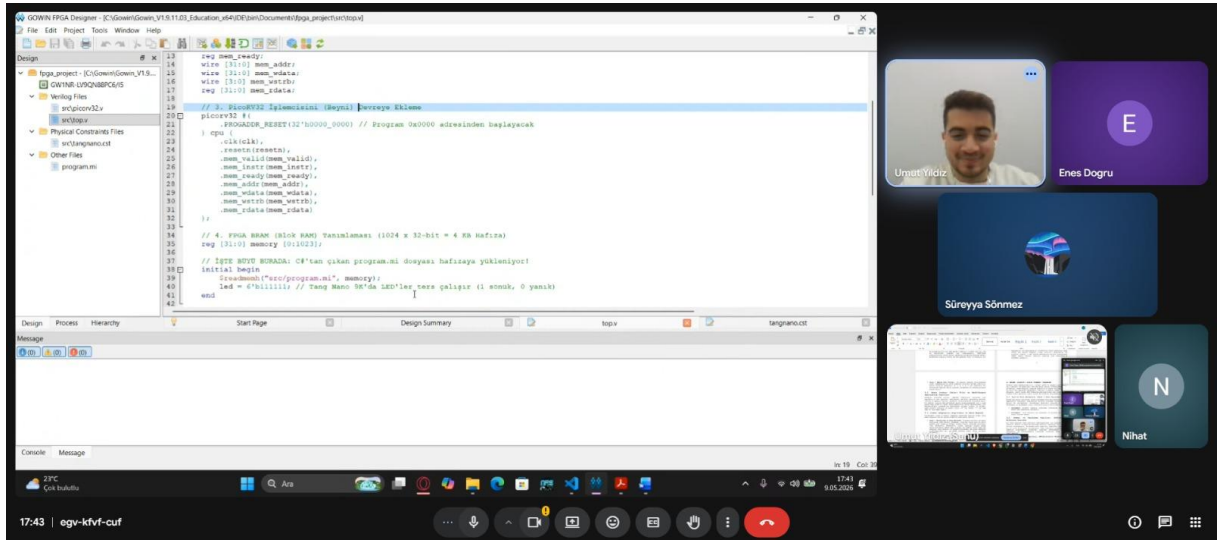
8.4. Sembol Çözümleme ve Arama Maliyeti

Problem: Linker'ın ikinci geçiş aşamasında, binlerce satırlık kod içindeki her bir modifikasyon kaydı için ESTAB üzerinde metin tabanlı (string) arama yapmak, derleme süresini ve sistem karmaşıklığını artırma potansiyeline sahipti.

Çözüm: Referans numaralandırma tekniği uygulanarak, harici semboller ReferenceTable isimli bir sözlükte tamsayı ID'ler ile eşleştirilmiştir. Modifikasyon işlemleri metinler üzerinden değil, doğrudan bu indeks numaraları üzerinden yapılarak O(1) karmaşıklığında hızlı bir çözümleme sağlanmıştır.

9. TAKIM ÇALIŞMASI VE PROJE YÖNETİMİ

Proje süresince takım üyeleri arasında etkili bir iş bölümü ve koordinasyon sağlanmıştır (PÇ12, PÇ13). Projenin Assembler genişletilmesi, Linker bellek haritası algoritmaları, nesne dosya yapılarının tasarımı ve FPGA entegrasyonu aşamaları eşzamanlı olarak paralel şekilde yürütülmüştür. Teknik dökümantasyonlar ve github açık kaynak kod repoları üzerinden iletişim teknikleri etkin kullanılarak takım içi bilgi paylaşımı en üst düzeyde tutulmuştur (PÇ12, PÇ13).



Şekil 12 Takım Çalışması

Tablo 2 Takım Görev Dağılımı

Takım Üyesi	Takım Görevi
Umutcan Yıldız	Rapor, Kod, FPGA Kart, Literatür.
Nihat Can Akkaya	Rapor, Kod, FPGA Kart, Literatür.
Süreyya Sönmez	Rapor, Kod, FPGA Kart, Literatür.
Enes Doğru	Rapor, Kod, FPGA Kart, Literatür.

10. SÜRDÜRÜLEBİLİRLİK VE TOPLUMSAL ETKİ

Bu projede geliştirdiğimiz Assembler ve Linker (Bağlayıcı) yazılımları, sadece akademik bir ödevin gerekliliklerini yerine getirmekle kalmayıp, sürdürülebilir ve topluma fayda sağlayan bir mühendislik ürünü olma hedefiyle tasarlanmıştır (PÇ8). Çözümümüzün uzun ömürlü (sürdürülebilir) olması için sistemi, belirli bir markaya veya donanımına bağımlı kalmayacak şekilde geliştirdik. Projemiz, FPGA dünyasında evrensel kabul gören standart veri okuma ve yazma kurallarına uygun çıktılar üreterek, donanım değişse bile yazılımın sorunsuz kullanılabilmesini sağlamaktadır.

Bunun yanında, bilginin paylaşımının toplumsal gelişimin temeli olduğuna inanarak projemizin kaynak kodlarını, hazırladığımız linker.ld script dosyalarını ve literatür testlerimizi GitHub platformu üzerinde herkesin erişimine açık hale getirdik [24]. Bu sayede;

- Bilgisayar mimarisi ve sistem programlama alanında çalışan diğer öğrenciler veya araştırmacılar, Linker işlemlerinin perde arkasında nasıl yürüdüğünü şeffaf bir şekilde inceleyebileceklerdir.
- JSON tabanlı kolay okunabilir nesne dosya formatımız, donanım tasarımıyla ilgilenen yeni mühendisler "kapalı ve karmaşık araçlar" yerine anlaşılır bir alternatif eğitim materyali sunacaktır.

Sonuç olarak projemiz; yaşam boyu öğrenme ilkesiyle (PÇ17) hareket edilerek mühendislik etiğine (PÇ9, PÇ10) tam uyumlu, açık kaynak standartlarını destekleyen sürdürülebilir bir değer haline getirilmiştir.

11. SONUÇ

Bu proje çalışmasıyla, PicoRV32 işlemci mimarisi için modüler bir yapıda tasarlanan iki geçişli linker sistemi başarıyla tamamlanmıştır. Projenin ana hedefi olan "birden fazla bağımsız nesne dosyasını birleştirme ve adresleri otomatik olarak bağlama" işlevi, geliştirilen C# tabanlı yazılım aracılığıyla literatür standartlarına uygun şekilde gerçekleştirilmiştir (PÇ1,PÇ6).

Çalışma sonucunda elde edilen kazanımlar ve ulaşılan hedefler şu şekilde özetlenebilir:

- **Mimarinin Başarısı:** Literatürde tanımlanan iki geçişli algoritma (Pass 1 ve Pass 2) başarıyla uygulanmış; dış sembollerin çözümlenmesi ve adres kaydırma (relocation) işlemleri için geliştirilen bit seviyesindeki matematiksel modellerin doğruluğu kanıtlanmıştır.
- **Donanım Uyumluluğu:** Hazırlanan makine zincirinin ürettiği makine kodları, Tang Nano 9K FPGA üzerinde test edilmiş; donanım ile yazılım arasındaki entegrasyonun sorunsuz çalıştığı fiziksel LED animasyonları ile doğrulanmıştır.
- **Akademik Doğrulama:** Geliştirilen sistemin çıktıları, hem Patterson & Hennessy gibi temel akademik kaynaklarla hem de bağımsız çevrimiçi simülatörlerle karşılaştırılarak evrensel RISC-V standartlarına uygunluğu belgelenmiştir (PÇ7).

Sonuç olarak bu proje, düşük seviyeli yazılım geliştirme ile donanım hiyerarşisi arasındaki doğrudan ilişkiyi kavramamızı sağlayan, ölçeklenebilir ve sürdürülebilir bir mühendislik deneyimi olmuştur. Geliştirilen bu altyapı, ileride eklenebilecek daha karmaşık komutlar veya kütüphaneler için sağlam bir temel teşkil etmektedir (PÇ17).

12. KAYNAKÇA VE ETİK UYUM

- [1] "Linker (computing)," Wikipedia, [Online]. Available: [https://en.wikipedia.org/wiki/Linker_\(computing\)](https://en.wikipedia.org/wiki/Linker_(computing))
- [2] J. R. Levine, *Linkers and Loaders*. San Francisco, CA, USA: Morgan Kaufmann, 2000.
- [3] L. L. Beck, *System Software: An Introduction to Systems Programming*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.
- [4] L. L. Beck, "Algorithm and Data Structures for a Linking Loader," in *System Software*.
- [5] "Executable and Linkable Format (ELF)," Wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Executable_and_Linkable_Format
- [6] "ELF Binaries on Linux: Understanding and Analysis," Linux Audit, [Online]. Available: <https://linux-audit.com/elf-binaries-on-linux-understanding-and-analysis/>
- [7] "Difference between Text file and Binary file," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/cpp/difference-between-cpp-text-file-and-binary-file/>
- [8] "Human Readability of Data Formats," PMC, NIH, [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11034916/>
- [9] "A Beginner's Guide to JSON," StackOverflow Blog, [Online]. Available: <https://stackoverflow.blog/2022/06/02/a-beginners-guide-to-json-the-data-format-for-the-internet/>
- [10] K. Cheng and J. Clarke, Eds., "RISC-V ABIs Specification," RISC-V International, [Online]. Available: <https://riscv-non-isa.github.io/riscv-elf-psabi-doc/>
- [11] "RISC-V ELF psABI Document," RISC-V International.
- [12] "LLVM: llvm::jitlink::riscv Namespace Reference," LLVM Project, [Online]. Available: https://llvm.org/doxygen/namespacellvm_1_1jitlink_1_1riscv.html
- [13] F. Song, "The Dark Side of RISC-V Linker Relaxation," MaskRay, [Online]. Available: <https://maskray.me/blog/2021-03-14-the-dark-side-of-riscv-linker-relaxation>
- [14] G. Stitt, "Organizing the Chaos of FPGA RAM Initialization: A Vendor- and Language-Agnostic Approach," Stitt Hub.
- [15] "Initialize Memory in Verilog," Project F, [Online]. Available: <https://projectf.io/posts/initialize-memory-in-verilog/>
- [16] "How to initialize contents of inferred block RAM (BRAM) in Verilog," StackOverflow.
- [17] N. Kišek, "A Linker for SIC/XE Hypothetical Computer," IPSI Transactions on Internet Research, vol. 15, no. 1, pp. 78-82, Jan. 2019.
- [18] S. Marz, "RARS (RISC-V Assembler and Runtime System)," UTK, [Online]. Available: <https://marz.utk.edu/my-courses/cosc230/rars/>

- [19] D. Camarmas-Alonso et al., "Venus: An Educational RISC-V Simulator," IEEE Access, 2024.
- [20] "LLD - The LLVM Linker," LLVM Project, [Online]. Available: <https://lld.llvm.org/>
- [21] "RVfpga: Understanding Computer Architecture," RISC-V International.
- [22] S. Bandara et al., "BRISC-V: An Open-Source Architecture Design Space Exploration Toolbox," Boston University.
- [23] "Trireme RISC-V Design Platform - A Complete Platform for RISC-V Design Space Exploration," [Online]. Available: <https://www.trireme-riscv.org/>
- [24] <https://github.com/RV32I-TwoPasserAssembler>

*Bilgiyi beceriyle
bütünleştiriyoruz*



Esentepe Kampüsü Serdivan/SAKARYA
T. 0 (264) 616 00 54 | F. 0 (264) 616 00 14
www.subu.edu.tr - bilgi@subu.edu.tr

